



Department of Industrial Management

Faculty of Science

University of Kelaniya

Centre of Excellence in Management and Information Technology

Assignment - Report

- Course Code - COST 21053
- Course Name – Object Oriented Programming (HU-22/23)
- Student Number : HS/2021/0051

Beacon Utility System (BUS)

B – Beacon : Refers to a guiding signal, representing the bus-tracking capability of the system.

U – Utility : Emphasizes the usefulness of the system for both tracking and ticketing.

S – System : Represents the complete, integrated solution for managing transportation needs.

Presentation Content

1. Problem Explanation
2. Purpose
3. DB Explanation
4. CRUD Operations
5. Interfaces Explanation
6. User Roles
7. Use of Main OOP Concepts
8. Exception Handling
9. Features to be add newly (not implemented)
10. Conclusion

Beacon Utility System

Problem Explanation

In the current world, there is a significant **problem** with **people using buses for transportation**. Sometimes, **ticket prices are suddenly increased without alerting passengers** And **illegal ticket prices** are calculated. Additionally passengers often do not know the bus arrival times at their current stop, **leading to missed buses**. On the other hand,when planning a trip, finding buses across multiple platforms can be **time-consuming**. this project solution addresses all these issues effectively.

Beacon Utility System

1. Purpose :

This software owned by a private bus transportation company. By using this application registered users can check,

- Bus routes
- Bus available times
- Ticket prices between start & stop destinations
- Bus arrival time to user nearest halt & stop destination
- Bus booking option for tours.

2. DB Explanation :

- Database name – BUS
- This database includes mainly 8 tables :
 1. NORMAL_USER – to store newly registered user details.
 2. PASSWORD – to store user passwords.
 3. BUS_INFO – this table stored company owned all buses details.
 4. TICKET_PRICE – to store updated ticket price.
 5. STAFF - this table stored staff details.
 6. ROUTE_1_TIMES – this table stored buses arrival time of each routes each bus halts
 7. LOST_ITEMS – to store lost item details & item photo
 8. BUS_RESERVATIONS – to store bus reservations details for tours

Beacon Utility System

3. CRUD Operations :

CREATE	READ	UPDATE	DELETE
1. New user registration process 2. Lost item adding process 3. Bus booking process	1. Password Checking process 2. Ticket price auto calculate process 3. Available buses & arrival time auto displaying process 4. Lost item displaying process 5. Image displaing process	1. Ticket price updating process 2. User details updating process 3. Password resetting process	1. User account removing with password process 2. Automatically removing lost items after 7 days.

4. Interfaces Explanation :

- This application includes 12 interfaces.
 1. **ADD_LOST_ITEMS** – to add lost items .
 2. **BOOK_BUS** – to reserve buses for tours.
 3. **CHECK_BUS** – to check bus routes, arrival times and ticket price.
 4. **DASHBOARD** – this dashboard shows options like book_bus,add lost item etc.
 5. **DisplayLostItems** – to display check lost items within 7 days.
 6. **DisplayReservations** – to check bus reservations (staff only).
 7. **LOGIN** – to log in to the system.
 8. **REG_NORMAL__USER** – to register as a normal user.
 9. **Main_Staff_Check**– to check staff before access.
 10. **RESET_PASSWORD** – to reset forgot passwords using NIC.
 11. **UPDATE_NORMAL_USER_1** – to update normal user details.
 12. **UPDATE_TICKET_PRICE** – to update new ticket prices (staff only).

Beacon Utility System

5. User Roles :

STAFF

1. Staff can update ticket prices (staff only)
2. Staff can check any available reservations (staff only)
3. Reset password using NIC

NORMAL USER

Normal user can,

1. Add user details
2. Choose start & stop Destinations
3. Reset password using NIC
4. Add lost items details with item photo
5. Remove user account using NIC
6. Choose bus id to check bus arrival times

6. Use of Main OOP Concepts :

- **Inheritance**

we used `javax.swing.JFrame` predefined Java Swing class as **parent class**, that provides a basic window framework, after we create **sub classes** using this parent class.

ex :

- `public class ADD_LOST_ITEMS extends javax.swing.JFrame { //code }`
- `public class DASHBOARD extends javax.swing.JFrame { //code }`
- `public class LOGIN extends javax.swing.JFrame { //code }`
-

(All interfaces inherit from this predefined swing super class)

6. Use of Main OOP Concepts :

- **Inheritance**

we created sub class named **Needmethod** inherit from **REG_NORMAL__USER** parent class. this **sub class** includes methods with combobox items, each method represent various item set.

ex :

```
public class Needmethod extends REG_NORMAL__USER{  
    abstract class A extends Needmethod{  
        //...more code available  
        public static void Kalutara(){  
            combox2.addItem("Panadura");  
            combox2.addItem("Horana");  
            combox2.addItem("Bandaragama");  
            combox2.addItem("Mathugama");  
            combox2.addItem("Beruwala");  
        }  
        //...more code available  
    }  
}
```

```
public class REG_NORMAL__USER extends javax.swing.JFrame {  
    //...more code available  
    if (selectedItem.equals("Panadura")) {  
        Needmethod.A.Panadura();  
    }  
    //...more code available  
}
```

Beacon Utility System

6. Use of Main OOP Concepts :

- **Encapsulation**

We created java class named DatabaseManager & encapsulate jdbcurl, username, password, connection & all processing methods. these data and processes hidden from the actual code in sub class named RESET_PASSWORD.

```
public class DatabaseManager {  
    // Encapsulated database connection details  
    private final String jdbcUrl;  
    private final String username;  
    private final String password;  
    private Connection connection;
```

After we create object in RESET_PASSWORD and send data to DatabaseManager.

```
DatabaseManager dbManager = new DatabaseManager(jdbcUrl, username, password);
```

and initialize private variable using setter method named

```
DatabaseManager(String jdbcUrl, String username, String password){  
    this.jdbcUrl = jdbcUrl;  
    this.username = username;  
    this.password = password;}  
}
```

6. Use of Main OOP Concepts :

- **Abstract**

we add abstract method named **populatecombobox**, to add item set using **most choosed bus routes**, but still we don't know about that most choosed bus routes, so we set this method as abstract for future updating process.

```
public class Needmethod extends REG_NORMAL__USER{  
  
    public abstract void populateComboBox();  
    //more code available  
}
```

6. Use of Main OOP Concepts :

- **Polymorphism**

- The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object. so we use this process to staff checking during ticket price change process & display reservations process example :

```
public class Main_Staff_Check extends javax.swing.JFrame {  
    String get_action;  
    public void set_action(String action){get_action = action;}  
    public void staff_check(){  
        //database password checking process code set available here..  
        //below codeset is update to be part in child class  
        new DisplayReservations().setVisible(true);  
    }  
}
```

Beacon Utility System

6. Use of Main OOP Concepts : Polymorphism

```
class PRICE_UPDATE_STAFF_CHECK extends Main_Staff_Check{
    @Override
    public void set_action(String action){
        get_action = action;
    }
    @Override
    public void staff_check(){
        //database password checking process code set available here(same as parent class)
        //below codeset is updated part(this part only)
        new UPDATE_TICKET_PRICE().setVisible(true);
    }
}
```

```
public class DASHBOARD extends javax.swing.JFrame {
    private void jButtonActionPerformed(java.awt.event.ActionEvent evt) {
        Main_Staff_Check rsc = new PRICE_UPDATE_STAFF_CHECK();
        rsc.set_action("change_price");
        rsc.setVisible(true); // Use the same instance
    }
}
```

UPDATE TICKET PRICE

Parent class

Child class

Beacon Utility System

6. Use of Main OOP Concepts : Polymorphism

```
public class DASHBOARD extends javax.swing.JFrame {  
    private void DISPLAY_LOST_ITEMS2ActionPerformed(java.awt.event.ActionEvent evt) {  
        Main_Staff_Check rsc = new Main_Staff_Check();  
        rsc.set_action("show_bookings");  
        rsc.setVisible(true); // Use the same instance  
    }  
}
```

BUS RESERVATIONS

Beacon Utility System

6. Use of Main OOP Concepts : Polymorphism

```
public class Main_Staff_Check extends javax.swing.JFrame {  
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {  
        String a = "show_bookings";  
        String b = "change_price";  
        if (get_action.equals(a)) {  
            staff_check(); // Call the parent class method  
        }  
        else if (get_action.equals(b)) {  
            Main_Staff_Check priceUpdate = new PRICE_UPDATE_STAFF_CHECK();  
            priceUpdate.OFFICERNIC = this.OFFICERNIC; // Share input fields  
            priceUpdate.OFFICERPW = this.OFFICERPW; // Share input fields  
            priceUpdate.staff_check(); // Call the overridden method  
        }  
    }  
}
```

Child class

Parent class

STAFF CHECK

ENTER OFFICER ID :

ENTER OFFICER PASSWORD :

CHECK

7. Exception Handling :

- We mostly use java,
 1. SQLException (for database processes)
 2. IOException (for file choosing process)
 3. ClassNotFoundException

With try – catch – finally – throws keywords.

8. Features to be add newly (not implemented) :

- Feature 01- new bus halts adding process
- Feature 02- new bus routes adding process
- Feature 03- change arrival times process

(These parts can change through database and java code.)

9. Conclusion

In conclusion, leveraging Java concepts, exception handling, SQL databases, and various Java constructs, we have successfully developed a user-friendly Java application to address transportation needs. This application, named Beacon Utility System (BUS), represents a reliable and efficient solution for modern transportation management.

Thank You